

A Geometric Nearest Point Algorithm for the Efficient Solution of the SVM Classification Task

Michael E. Mavroforakis, Margaritis Sdralis, and
Sergios Theodoridis

Abstract—Geometric methods are very intuitive and provide a theoretically solid approach to many optimization problems. One such optimization task is the support vector machine (SVM) classification, which has been the focus of intense theoretical as well as application-oriented research in machine learning. In this letter, the incorporation of recent results in reduced convex hulls (RCHs) to a nearest point algorithm (NPA) leads to an elegant and efficient solution to the SVM classification task, with encouraging practical results to real-world classification problems, i.e., linear or nonlinear and separable or nonseparable.

Index Terms—Classification, kernel methods, nearest point algorithm (NPA), pattern recognition, reduced convex hulls (RCHs), support vector machines (SVMs).

I. INTRODUCTION

Support vector machines (SVMs) have by now been established as an indispensable tool for pattern recognition and regression tasks, e.g., [1] and [2]. A major research topic, related to SVMs, is focused on their computational aspects. One of the most popular algorithms to obtain the SVM solution is the sequential minimal optimization (SMO) algorithm [3] and its variants [4]. More recently [5], [6], it was demonstrated that significant computational savings can be obtained by exploiting the geometric interpretation of SVMs, as pointed out originally in [7] and [8]. Building upon the ν -SVM formulation ([9]), it was proved that the SVM optimal solution, for the nonseparable class case, is equivalent with solving the nearest point problem between the reduced convex hulls (RCHs) associated with the two classes. An algorithm that solves the nearest point problem for the case of RCHs was first presented in [5] and [6]. That algorithm was based on Schlesinger–Kozinec’s (S–K) algorithm, originally developed for convex hulls. In this letter, a new geometric algorithm is derived, inspired by the classical Gilbert’s algorithm [10]. The new algorithm turns out to be computationally more efficient compared to the algorithms derived in [5] and [6]. Moreover, the new algorithm depends only on inner product operations and, therefore, is appropriate for the more general nonseparable and nonlinear class tasks.

In Section II, a brief introduction to the SVM paradigm is given and the notion of RCHs is provided. In Section III, the novel geometric algorithm is presented and, in Section IV, comparative results of this algorithm are given for a number of known data sets used in the literature.

II. RCHS AND THE GEOMETRIC VIEW OF SVMs

The reduced convex hull of a set of points X , denoted as $R(X, \mu)$, $\mu < 1$, is defined as the set

$$R(X, \mu) = \left\{ w : w = \sum_{i=1}^k a_i x_i, x_i \in X, \sum_{i=1}^k a_i = 1, 0 \leq a_i \leq \mu \right\}.$$

Manuscript received November 24, 2006; revised March 2, 2007; accepted March 25, 2007.

The authors are with the Informatics and Telecommunications Department, University of Athens, Athens 15771, Greece (e-mail: mmavrof@di.uoa.gr; msdralis@di.uoa.gr; stheodor@di.uoa.gr).

Digital Object Identifier 10.1109/TNN.2007.900237

The only difference with the definition of a convex hull is that the α_i ’s are now upper bounded by $\mu < 1$. It can be shown that $R(X, \mu_1) \subseteq R(X, \mu_2)$, $\mu_1 \leq \mu_2$. Thus, by choosing a small enough value for μ , one can make the respective RCHs of the two classes to be separable, provided that the mean values of the two classes do not coincide, e.g., [5]. It has been shown [7], [8] that the segment joining the nearest points between RCHs defines a hyperplane that points in the same direction as the one resulting from the respective SVM solution.

Although computing the nearest points between convex hulls is a rather well-studied problem, it turns out not to be so for the case of RCHs. The nearest point algorithms rely on computing extreme points of the respective convex hulls or, alternatively, their projections on a given direction. In the case of the convex hull of a set X , each extreme point coincides with a point in X . However, this is not the case with the extreme points in an RCH. As it has been shown in [5], each extreme point of an RCH is a combination of $\lceil 1/\mu \rceil$ points in X . This leads to a problem of combinatorial nature and complexity rockets up as μ gets smaller. To bypass this difficulty, a theorem is shown in [5] that leads to a computationally simple procedure that computes the minimum projection from all the points in an RCH on a given direction (which actually corresponds to an extreme RCH point). It turns out that this is the only information needed, for each iteration step, by a number of nearest point algorithms. The minimum projection procedure is outlined as follows for the sake of clarity.

- 1) Obtain the projections of the original points onto the given direction.
- 2) Sort these projections in ascending order and choose the $\lceil 1/\mu \rceil$ first of them.
- 3) Take the weighted average of the selected ordered projections, with weight μ for the $\lceil 1/\mu \rceil$ smallest projections and $1 - \lceil 1/\mu \rceil \mu$ (if it is nonzero) for the last ($\lceil 1/\mu \rceil$ th) projection. The result is the projection of the extreme RCH point, required in the algorithmic computations. Note also that knowing the projection, the respective extreme point can easily be derived, if it is required.

In the sequel, we will use this setting to exploit the fast and intuitive converging properties of Gilbert’s algorithm [10] and develop its counterpart for the case of RCHs.

III. GEOMETRIC ALGORITHM FOR SVM SEPARABLE AND NONSEPARABLE TASKS

Using the RCH framework, the general (nonseparable) SVM classification task can be formulated as follows [7], [11]: Given a set of patterns $\{x_i\}$ and their corresponding labels $\{y_i\}$, $y_i \in \{-1, 1\}$, where $I_1 = \{i : y_i = 1\}$, $I_2 = \{i : y_i = -1\}$, $X_1 = \{x_i : i \in I_1\}$, and $X_2 = \{x_i : i \in I_2\}$, find the couple¹ of points (x_1^*, x_2^*) , such that

$$(x_1^*, x_2^*) = \arg \min (\|x_1 - x_2\|) \quad (1)$$

where $x_1 \in R(X_1, \mu)$ and $x_2 \in R(X_2, \mu)$, assuming that the parameter μ has been selected properly to guarantee that $R(X_1, \mu) \cap R(X_2, \mu) = \emptyset$. This is clearly a nearest point problem (NPP) and it is equivalent to the following minimum norm problem (MNP) of the Minkowski difference set [4]: find w^* such that

$$w^* = \arg \min_{z \in Z} (\|z\|) \quad (2)$$

¹There may be more than one such couple of points, in case of degeneracy, but the distance and direction between the points of each couple will be the same for all couples, thus leading to a unique solution.

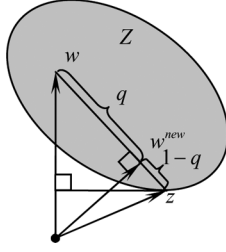


Fig. 1. Elements involved in Gilbert's algorithm.

where $Z = \{z : z = x_1 - x_2, x_1 \in R(X_1, \mu), x_2 \in R(X_2, \mu)\}$. Obviously, the restriction that the RCHs do not overlap means, equivalently, that $\|w^*\| > 0$ (null vector does not belong to Z).

A brief description of the standard Gilbert's algorithm, (provided that Z is a convex set, of which we need to find the minimum norm member z^*), is given in the following.

Step 1: Choose $w \in Z$.

Step 2: Find the point $z \in Z$ with the minimum projection onto the direction of w . If $\|w\| \cong \|z\|$, set $w^* \leftarrow z$; stop.

Step 3: Find the point w^{new} of the line segment $[w, z]$, with minimum norm (closest to the origin). Set $w \leftarrow w^{\text{new}}$ and go to Step 2.

The idea behind the algorithm is very simple and intuitive and the elements involved in the previously described steps of the algorithm are illustrated in Fig. 1. It has been proven that such a scheme converges to the minimum norm solution [10].

Theorem 2 of [5] provides an efficient (noncombinatorial) way of performing the necessary adaptations.

Let $w \in Z$ and $z \in Z$ such that

$$z = \arg \min_{u \in Z} \left(\frac{\langle u, w \rangle}{\|w\|} \right) \quad (3)$$

i.e., the point of the convex set Z that exhibits the least projection onto the direction of w . According to Gilbert's algorithm, we have to find the point w^{new} of the line segment $[w, z]$ with the least norm and use it for the next iteration of the algorithm. Assuming $\|w\| \neq \|z\|$ (otherwise, the algorithm would have stopped from Step 2), it is easily verified (using the *projection theorem* [12]) that

$$w^{\text{new}} = (1 - q)w + qz, \quad q \in (0, 1] \quad (4)$$

where

$$q = \begin{cases} s = \frac{(\|w\|^2 - \langle w, z \rangle)}{\|z - w\|^2}, & s \in (0, 1] \\ 1, & \text{otherwise} \end{cases} \quad (5)$$

For the calculation of q (and, hence, of w^{new}), the quantities $\langle w, w \rangle$, $\langle w, z \rangle$, and $\langle z, z \rangle$ need to be calculated. Since $w = w_1 - w_2$, where $w_1 \in R(X_1, \mu)$ and $w_2 \in R(X_2, \mu)$, we have

$$\langle w, w \rangle = \langle w_1, w_1 \rangle + \langle w_2, w_2 \rangle - 2\langle w_1, w_2 \rangle. \quad (6)$$

By the definition of an RCH, we can express w as $w = \sum_{i \in I_1} \alpha_i x_i - \sum_{i \in I_2} \alpha_i x_i$ or

$$w = \sum_{i \in I_1 \cup I_2} a_i y_i x_i, \quad 0 \leq a_i \leq \mu \quad (7)$$

$$\sum_{i \in I_k} a_i = 1, \quad k \in \{1, 2\}.$$

The detailed calculations of the quantities involved are presented in the Appendix. Observe that the calculations are as follows: 1) distinct from those referred in the standard form of the Gilbert's algorithm, since RCHs are now involved and 2) dependent *only on inner products* of the original points and not the points themselves, i.e., the algorithm is directly kernelizable.

Finally, since the *optimal hyperplane* is defined by

$$\langle w^*, x \rangle + b^* = 0 \quad (8)$$

where $w^* = w_1^* - w_2^*$, setting $x = (w_1^* + w_2^*)/2$, the threshold is

$$b^* = \frac{(\langle w_2^*, w_2^* \rangle) - \langle w_1^*, w_1^* \rangle}{2}. \quad (9)$$

To transform the previous relations to the *canonical form* (w^c, b^c) (i.e., the point(s) z closest to the hyperplane satisfy $|\langle w^c, z \rangle + b^c| = 1$ [13]), we have to rescale (w^*, b^*) by a scaling factor $\eta > 0$. Omitting the intermediate calculations

$$\eta = \frac{2}{\|w^*\|^2} \quad (10)$$

$$w^c = \frac{2(w_1^* - w_2^*)}{\|w_1^* - w_2^*\|^2} \quad (11)$$

$$b^c = \frac{(\langle w_2^*, w_2^* \rangle) - \langle w_1^*, w_1^* \rangle}{\|w_1^* - w_2^*\|^2}. \quad (12)$$

Accordingly, since $w_1^* = \sum_{i \in I_1} a_i^* x_i$ and $w_2^* = \sum_{i \in I_2} a_i^* x_i$, we obtain

$$a_i^c = \eta a_i^*. \quad (13)$$

The value b^c in (12) represents the offset of the hyperplane bisecting the line segment joining the couple of closest points of the two RCHs, each constructed from the corresponding data set. However, this offset usually does not coincide with the one that results from the algebraic optimization problem, as it is pointed out in [8]. To reach the algebraically optimal solution, the hyperplane must be shifted by a quantity proportional to the reduction bound μ and the signed sum of the slack variables ξ_i . Thus, the optimal offset value b^{oc} is

$$b^{oc} = b^c - \frac{\mu}{2} \sum_{i \in I_1 \cup I_2} y_i \xi_i \quad (14)$$

where it is calculated as follows [11], [14]:

$$\xi_i = \begin{cases} 1 - y_i(\langle w^c, x_i \rangle + b^c), & \text{if } 1 - y_i(\langle w^c, x_i \rangle + b^c) > 0 \\ 0, & \text{if } 1 - y_i(\langle w^c, x_i \rangle + b^c) \leq 0 \end{cases} \quad (15)$$

A very brief but clear and intuitive explanation of this difference is that, for the same value of the bound μ , the convex hulls are not reduced by the same factor; their reduction factor actually depends, not only on μ , but also on the number and the distribution of the samples.

According to the aforementioned and the calculations given in the Appendix, the novel algorithm is summarized as follows.

Step 1: Set the initial points w_1 and w_2 (actually their corresponding multipliers a_i) to be any points belonging to the respective RCHs, e.g., the centroid.

Step 2: Find the point $z \in Z$ with the minimum projection onto the direction of w . This point is defined in (3) and can be computed from (25), (6), and (16)–(18). Note that this point is expressed as a weighted average of the original points x_i . Compare the norms $\|w\|$ and $\|z\|$; if the stopping criterion ($\|w\| \cong \|z\|$) is not met, go to Step 3; else, compute b [through (36) and (14)] and exit.

TABLE I
DETAILS OF THE DATA SETS USED FOR TESTING AND THE CORRESPONDING
SUCCESS RATE REPORTED IN THE LITERATURE

Dataset	Data Dim.	Train Patterns	Test Patterns	Success Reported
Diabetis	8	468	300	76.5±1.7
Banana	2	400	4900	88.5±0.7
German	20	700	300	76.4±2.1
Wave-form	21	400	4600	90.1±0.4
Flare-Solar	9	666	400	67.6±1.8
Heart	13	170	100	84.0±3.3

TABLE II
PARAMETERS OF THE KERNEL MODEL USED (RBF KERNEL HAS BEEN USED
EVERYWHERE) AND TUNING PARAMETERS FOR THE SPECIFIC ALGORITHMS
USED TO ACHIEVE THE REPORTED SUCCESS RATE

Dataset	σ	C , tolerance	μ , accuracy
Diabetis	20	100, 10^{-3}	0.0075, 10^{-3}
Banana	1	316.2, 10^{-3}	0.014, 5×10^{-4}
German	55	3162, 10^{-3}	0.0052, 10^{-3}
Wave-form	20	1000, 10^{-3}	0.02, 5×10^{-4}
Flare-Solar	30	1000, 10^{-3}	0.0039, 10^{-3}
Heart	120	1000, 10^{-3}	0.017, 10^{-3}

Step 3: Find the point w^{new} of the line segment $[w, z]$ with minimum norm (closest to the origin). Set $w \leftarrow w^{\text{new}}$. This corresponds to the update of the coefficients a_i [using (5) for the calculation of q and (33) and (34)]. Go to Step 2.

During the previously described steps, intermediate quantities can be introduced, computed, and updated (according to the formulas in the Appendix) to speed up the calculations.

IV. EXPERIMENTS

In order to extensively investigate the performance (both in speed and accuracy) of the new algorithm presented here, a number of publicly available test data sets, with known success rates under specific SVM training models (referred in the literature) have been used.

Four different SVM classification algorithms have been implemented, tested, and compared to the new algorithm (denoted as RCH-G): the SMO algorithm, as it was presented by Platt in [3] (denoted as SMO), the two modified SMO algorithms presented by Keerthi *et al.* in [15] (denoted as SMO-K1 and SMO-K2), and the S-K geometric algorithm (denoted by RCH-SK), presented in [5] and [6]. Each algorithm was trained and tested for each data set, under the same model (kernel with the corresponding parameters) in order to achieve the same accuracy referred in the literature [16], [17]. The accuracies of all classifiers, achieved for each specific data set, were calculated under the same validation scheme, i.e., the same validation method and the same data realizations (100 publicly available realizations [16] of the data sets). The standard data sets used were the following: 1) diabetis, 2) banana, 3) German, 4) waveform, 5) flare-solar, and 6) heart. The details of these data sets as well as the success rate reported in the literature are presented in Table I; the used parameters are shown in Table II; and the results of the runs are summarized in Table III.

The results of the new geometric algorithm presented here, compared to the most popular and fast algebraic ones, are very encouraging. Its advantage with respect to the number of kernel evaluations, *for the same level of accuracy*, compared to the most popular algebraic tech-

TABLE III
RESULTS ACHIEVED FOR EACH ALGORITHM (SUCCESS RATE AND NUMBER OF
KERNEL EVALUATIONS). BEST VALUES ARE IN BOLD

Dataset	Method	Success (%)	Kernel Eval. ($\times 10^6$)
Diabetis	SMO	76.7±1.8	20.10
	SMO-K1	76.7±1.8	1.45
	SMO-K2	76.7±1.8	4.10
	RCH-SK	76.2±1.8	0.14
	RCH-G	76.3±1.8	0.70
Banana	SMO	89.1±0.6	564.33
	SMO-K1	89.1±0.6	6.60
	SMO-K2	89.0±0.7	15.40
	RCH-SK	89.0±0.6	14.27
	RCH-G	88.8±0.9	1.75
German	SMO	76.0±2.2	13.70
	SMO-K1	76.1±2.2	9.00
	SMO-K2	76.1±2.2	46.20
	RCH-SK	75.5±0.2	3.62
	RCH-G	76.3±0.4	2.71
Waveform	SMO	88.8±0.6	94.30
	SMO-K1	89.2±0.5	2.20
	SMO-K2	88.9±1.4	6.20
	RCH-SK	87.8±2.3	3.81
	RCH-G	88.5±0.7	1.46
Flare-Solar	SMO	67.5±1.8	22.80
	SMO-K1	67.6±1.8	11.90
	SMO-K2	67.6±1.8	18.60
	RCH-SK	67.6±1.7	2.08
	RCH-G	67.7±1.8	3.37
Heart	SMO	83.9±3.3	2.05
	SMO-K1	83.2±4.2	0.26
	SMO-K2	83.4±4.2	0.26
	RCH-SK	84.2±2.7	7.50
	RCH-G	84.8±3.3	0.05

niques, is readily noticeable. The enhanced performance is justified by the fact that, although the algebraic algorithms (especially SMO-K2) make a clever utilization of the cache, where kernel values are stored, they cannot avoid repetitive searches in order to find the best couple of points (working set selection) to be used in the next iteration of the optimization process. Furthermore, the enhanced performance of the new geometric algorithm against its algebraic competitors can be explained from the fact that it is a straightforward optimization algorithm, with a clear optimization target at each iteration step, always aiming at the global minimum and at the same time being independent of obscure and sometimes inefficient heuristics. Compared to the other existing geometric algorithm RCH-SK presented in the literature [5], [6], it exhibits equivalent or better speed results, thus ranking this algorithm among the fastest and at the same time the simplest and most intuitive ones for the solution of the general SVM classification task.

V. CONCLUSION

The SVM approach to machine learning has both theoretical and practical advantages over a number of alternatives. The geometric intuition is intrinsic to the structure of SVM and has found application in solving both the separable and nonseparable problem. Gilbert's iterative geometric algorithm for the NPP, modified here to accommodate the RCH notion for the nonseparable task, resulted in a very promising method of solving SVM. The obtained results (in terms of kernel evaluations) showed a clear advantage when compared with the best representatives of the algebraic (decomposition) methods, i.e., SMO and its

improved variants, and at least equivalent value to the existing RCH-SK geometric algorithm. The algorithm presented here is based on clear and sound geometric concepts, does not use any heuristics, and provides a clear understanding of the convergence process and the role of the parameters used.

APPENDIX

In the sequel, the calculations of the quantities involved in the novel algorithm will be presented. Let

$$\langle w_1, w_1 \rangle = \sum_{i \in I_1} \sum_{j \in I_1} a_i a_j \langle x_i, x_j \rangle \equiv A \quad (16)$$

$$\langle w_2, w_2 \rangle = \sum_{i \in I_2} \sum_{j \in I_2} a_i a_j \langle x_i, x_j \rangle \equiv B \quad (17)$$

$$\langle w_1, w_2 \rangle = \sum_{i \in I_1} \sum_{j \in I_2} a_i a_j \langle x_i, x_j \rangle \equiv C. \quad (18)$$

Recall [according to (3)] that we seek an element $z \in Z$ which minimizes the quantity $\langle w, u \rangle / \|w\|$ (which, in turn, corresponds to the projection of $u \in Z$ onto the direction of w). For a given w , it suffices to search for the minimizer of the quantity $\langle w, u \rangle$. It is

$$\min \langle w, u \rangle = \min \langle w_1 - w_2, u_1 \rangle + \min \langle w_2 - w_1, u_2 \rangle. \quad (19)$$

Let $r \in I_1 \cup I_2$ and set

$$\langle x_r, w_1 \rangle = \sum_{i \in I_1} a_i \langle x_i, x_r \rangle \equiv D_r \quad (20)$$

$$\langle x_r, w_2 \rangle = \sum_{i \in I_2} a_i \langle x_i, x_r \rangle \equiv E_r. \quad (21)$$

Accordingly

$$\langle w_1 - w_2, x_r \rangle = \left\langle x_r, \sum_{i \in I_1} a_i x_i - \sum_{j \in I_2} a_j x_j \right\rangle = D_r - E_r. \quad (22)$$

The minimizers z_1 and z_2 of (19) are constructed (according to [5, Th. 2]) from the points of least projections onto $w_1 - w_2$ and $w_2 - w_1$, respectively. Therefore, we order the differences as follows:

$$\tilde{S}^1 \equiv \{D_{r_1} - E_{r_1}, \dots, D_{r_{m_1+1}} - E_{r_{m_1+1}}\},$$

$$r \in I_1; \quad i < j \Rightarrow D_{r_i} - E_{r_i} \leq D_{r_j} - E_{r_j} \quad (23)$$

$$\tilde{S}^2 \equiv \{E_{r_1} - D_{r_1}, \dots, E_{r_{m_2+1}} - D_{r_{m_2+1}}\},$$

$$r \in I_2; \quad i < j \Rightarrow E_{r_i} - D_{r_i} \leq E_{r_j} - D_{r_j} \quad (24)$$

and set $\tilde{I}_1$ and $\tilde{I}_2$ as the sets of the indices of $\tilde{S}^1$ and $\tilde{S}^2$, which are subsets of I_1 and I_2 , respectively. Consequently, denoting by $I(1 : \text{end})$ all the (ordered) elements of set I , (19) becomes

$$\min \langle w, z \rangle = \mu \sum_{i \in \tilde{I}_1(1:\text{end}-1)} \tilde{S}_i^1 + \lambda \tilde{S}_{\tilde{I}_1(\text{end})}^1$$

$$+ \mu \sum_{j \in \tilde{I}_2(1:\text{end}-1)} \tilde{S}_j^2 + \lambda \tilde{S}_{\tilde{I}_2(\text{end})}^2 \quad (25)$$

where $\lambda = 1 - \lfloor 1/\mu \rfloor \mu$ and the corresponding z_1 and z_2 are

$$z_1 = \mu \sum_{i \in \tilde{I}_1(1:\text{end}-1)} x_i + \lambda x_{\tilde{I}_1(\text{end})} \quad (26)$$

$$z_2 = \mu \sum_{i \in \tilde{I}_2(1:\text{end}-1)} x_i + \lambda x_{\tilde{I}_2(\text{end})}. \quad (27)$$

Therefore, $\langle z_1, z_1 \rangle$ (and, similarly, $\langle z_2, z_2 \rangle$ and $\langle z_1, z_2 \rangle$) is

$$\langle z_1, z_1 \rangle = \mu^2 \sum_{i \in \tilde{I}_1(1:\text{end}-1)} \sum_{j \in \tilde{I}_1(1:\text{end}-1)} \langle x_i, x_j \rangle$$

$$+ 2\mu\lambda \sum_{j \in \tilde{I}_1(1:\text{end}-1)} \langle x_{\tilde{I}_1(\text{end})}, x_j \rangle$$

$$+ \lambda^2 \langle x_{\tilde{I}_1(\text{end})}, x_{\tilde{I}_1(\text{end})} \rangle. \quad (28)$$

From (26)–(28), it is easy to calculate $\langle z, z \rangle$

$$\langle z, z \rangle = \langle z_1, z_1 \rangle + \langle z_2, z_2 \rangle - 2\langle z_1, z_2 \rangle. \quad (29)$$

Next, we have to calculate the new coefficients a_i^{new} . It is

$$w^{\text{new}} = (1 - q)w + qz \Leftrightarrow \quad (30)$$

$$w_1^{\text{new}} = (1 - q)w_1 + qz_1 \quad (31)$$

and

$$w_2^{\text{new}} = (1 - q)w_2 + qz_2. \quad (32)$$

Consequently, if $i \in I_1$, from (26) through (31), it is

$$a_i^{\text{new}} = (1 - q)a_i + q\mu \sum_{j \in \tilde{I}_1(1:\text{end}-1)} \delta_{ij} + q\lambda \sum_{j \in \tilde{I}_1(\text{end})} \delta_{ij}. \quad (33)$$

Similarly, if $i \in I_2$, from (32), it is

$$a_i^{\text{new}} = (1 - q)a_i + q\mu \sum_{j \in \tilde{I}_2(1:\text{end}-1)} \delta_{ij} + q\lambda \sum_{j \in \tilde{I}_2(\text{end})} \delta_{ij}. \quad (34)$$

The quantities b^* of (9) and b^c of (12) become, respectively

$$b^* = \frac{1}{[2(B - A)]} \quad (35)$$

$$b^c = \frac{(B - A)}{(A + B - 2C)}. \quad (36)$$

As a last step, the new values of A , B , C , D_i , and E_i have to be found. Only A^{new} and D_r^{new} follow (the rest are similar):

$$A^{\text{new}} = \langle w_1^{\text{new}}, w_1^{\text{new}} \rangle = (1 - q)^2 A$$

$$+ (q\mu)^2 \sum_{i \in I_1} \sum_{j \in I_1} \sum_{k \in \tilde{I}_1(1:\text{end}-1)} \sum_{l \in \tilde{I}_1(1:\text{end}-1)} \delta_{ik} \delta_{jl} \langle x_i, x_j \rangle$$

$$+ (q\lambda)^2 \sum_{i \in I_1} \sum_{j \in I_1} \sum_{k \in \tilde{I}_1(\text{end})} \sum_{l \in \tilde{I}_1(\text{end})} \delta_{ik} \delta_{jl} \langle x_i, x_j \rangle$$

$$+ 2(1 - q)q\mu \sum_{i \in I_1} \sum_{j \in I_1} \sum_{k \in \tilde{I}_1(1:\text{end}-1)} \delta_{jk} a_i \langle x_i, x_j \rangle$$

$$+ 2(1 - q)q\lambda \sum_{i \in I_1} \sum_{j \in I_1} \sum_{l \in \tilde{I}_1(\text{end})} \delta_{jl} a_i \langle x_i, x_j \rangle$$

$$+ 2q^2\mu\lambda \sum_{i \in I_1} \sum_{j \in I_1} \sum_{k \in \tilde{I}_1(1:\text{end}-1)} \sum_{l \in \tilde{I}_1(\text{end})} \delta_{ik} \delta_{jl} \langle x_i, x_j \rangle \quad (37)$$

and

$$D_r^{\text{new}} = \langle x_r, w_1^{\text{new}} \rangle = (1 - q)D_r + q\mu \sum_{i \in I_1} \sum_{j \in \tilde{I}_1(1:\text{end}-1)} \delta_{ij} \langle x_r, x_i \rangle$$

$$+ q\lambda \sum_{i \in I_1} \sum_{j \in \tilde{I}_1(\text{end})} \delta_{ij} \langle x_r, x_i \rangle. \quad (38)$$

REFERENCES

- [1] V. N. Vapnik, *Statistical Learning Theory*. New York: Wiley, 1998.
- [2] S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 3rd ed. New York: Academic, 2006.
- [3] J. Platt, "Fast training of support vector machines using sequential minimal optimization," in *Advances in Kernel Methods—Support Vector Learning*, B. Schölkopf, C. Burges, and A. Smola, Eds. Cambridge, MA: MIT Press, 1999, pp. 185–208.
- [4] S. S. Keerthi, S. K. Shevade, C. Bhattacharyya, and K. R. K. Murthy, "A fast iterative nearest point algorithm for support vector machine classifier design," Dept. CSA, IISC, Bangalore, India, Tech. Rep. TR-ISL-99-03, 1999.
- [5] M. Mavroforakis and S. Theodoridis, "A geometric approach to support vector machine (SVM) classification," *IEEE Trans. Neural Netw.*, vol. 17, no. 3, pp. 671–682, May 2006.
- [6] Q. Tao, G.-W. Wu, and J. Wang, "A generalized S-K-algorithm for learning ν -SVM classifiers," *Pattern Recognit. Lett.*, vol. 25, no. 10, pp. 1165–1171, 2004.
- [7] K. P. Bennett and E. J. Bredensteiner, "Geometry in learning," in *Geometry at Work*, C. Gorini, E. Hart, W. Meyer, and T. Phillips, Eds. Washington, DC: Mathematical Association of America, 1998.
- [8] D. J. Crisp and C. J. C. Burges, "A geometric interpretation of ν -SVM classifiers," in *Advances in Neural Information Processing Systems*, S. Solla, T. Leen, and K.-R. Müller, Eds. Cambridge, MA: MIT Press, 2000, vol. 12, pp. 244–250.
- [9] B. Schölkopf, A. J. Smola, R. C. Williamson, and P. L. Bartlett, "New support vector algorithms," *Neural Comput.*, vol. 12, pp. 1207–1245, 2000.
- [10] E. Gilbert, "Minimizing the quadratic form on a convex set," *SIAM J. Control*, vol. 4, pp. 61–79, 1966.
- [11] D. Zhou, B. Xiao, H. Zhou, and R. Dai, "Global geometry of SVM classifiers," AI Lab, Inst. Autom., Chinese Academy, Tech. Rep. 30-5-02, 2002.
- [12] D. G. Luenberger, *Optimization by Vector Space Methods*. Wiley, 1969.
- [13] B. Schölkopf and A. Smola, *Learning With Kernels—Support Vector Machines, Regularization Optimization and Beyond*. Cambridge, MA: MIT Press, 2002.
- [14] K. P. Bennett and E. J. Bredensteiner, P. Langley, Ed., "Duality and geometry in SVM classifiers," in *Proc. 17th Int. Conf. Mach. Learn.*, 2000, pp. 57–64.
- [15] S. Keerthi, S. Shevade, C. A. Bhattacharyya, and K. Murthy, "Improvements to Platt's SMO algorithm for SVM classifier design," Dept. CSA, IISC, Bangalore, India, Tech. Rep., 1999.
- [16] G. Rätsch, T. Onoda, and K.-R. Müller, "Soft margins for AdaBoost," *Mach. Learn.*, vol. 42, no. 3, pp. 287–320, 2001.
- [17] S. Mika, G. Rätsch, J. Weston, B. Schölkopf, and K.-R. Müller, "Fisher discriminant analysis with kernels," in *Neural Networks for Signal Processing IX*, Y. Hu, J. Larsen, E. Wilson, and S. Douglas, Eds. Piscataway, NJ: IEEE Press, 1999, pp. 41–48.